

regex_replace [SimTalk]

Searches for the specified *Text* in the passed text that matches the *regular expression*.

Remarks

regex_replace then replaces all *strings*, which match the *RegularExpression*, with the text you specify as the *ReplacingText*.

Note

To search for a backslash (\) and replace it with another character, type in four backslashes, \\\\.

This is because the *SimTalk* parser uses the backslash as an **escape character**.

```
// replaces the backslash with an empty space
regex_replace("my\\text", "\\\\", " ")

print regex_replace("my\\text", "\\\\", " ")
-- outputs my text
```

Type

Function

Syntax

```
regex_replace(Text:string, RegularExpression:string, ReplacingText:string)
→ string
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text which you want to search for the *regular expression*.
- The parameter *RegularExpression* of data type *string* designates the *regular expression* that you want to replace.
- The parameter *ReplacingText* of data type *string* designates the text that is to replace the *regular expression*.

Return Value

The return value has the data type *string*.

It is the text that replaced the original *regular expression*.

Examples

```
// replaces the time by the hour
regex_replace("1234567 9:45pm 1234567 7:45pm 1234", "((1[0-2])|(0?[1-9])):
([0-5][0-9])((am)|(pm))", "$1")
// results in "1234567 9 1234567 7 1234"
```

```
// Removes all space characters
print regex_replace("This is a test", " ", "")
// Results in Thisisatest
```

```
// Replaces the characters /, : and space by the character |
print regex_replace("2018/09/12 12:33", "[/: ]", "|")
// Results in 2018|09|12|12|33
```

See also

[regex_search \[SimTalk\]](#)

[regex_search2 \[SimTalk\]](#)

regex_search [SimTalk]

Searches for the specified *Text* in the passed text that matches the *RegularExpression*.

Remarks

A *regular expression* is a sequence of characters that define a search pattern, mainly for use in **pattern matching** with strings, or **string matching**, i.e. **find and replace**-like operations.

Each character in a regular expression is either understood to be a metacharacter with its special meaning, or a regular character with its literal meaning. Together, they can be used to identify textual material of a given pattern, or process a number of instances of it that can vary from a precise equality to a very general similarity of the pattern. The pattern sequence itself is an expression that is a statement in a language designed specifically to represent prescribed targets in the most concise and flexible way to direct the automation of text processing of general text files, specific text forms, or of random input strings.

A very simple use of a regular expression would be to locate the same word spelled two different ways in a text editor, for example the regular expression **seriali[sz]e** matches both **serialise** and **serialize** (quoted after https://en.wikipedia.org/wiki/Regular_expression).

Metacharacter	Description	Example
.	Matches any single character. Within bracket expressions, the dot character matches a literal dot.	a.c matches " abc ", etc., but [a.c] matches only " a ", " . ", or " c ".
?	Matches the preceding element zero or one time.	ab?c matches only " ac " or " abc ".
?	Modifies the *, +, ? or {m,n}'d regular expression that comes before to match as few times as possible.	—
*	Matches the preceding element zero or more times.	ab*c matches " ac ", " abc ", " abbbc ", etc. [xyz]* matches "", " x ", " y ", " z ", " zx ", " zyx ", " xyzy ", and so on. (ab)* matches "", " ab ", " abab ", " ababab ", and so on.
+	Matches the preceding element one or more times.	ab+c matches " abc ", " abbc ", " abbbc ", and so on, but not " ac ".
{m,n}	Matches the preceding element at least m and not more than n times.	a{3,5} matches only " aaa ", " aaaa ", and " aaaaa ". This is not found in a few older instances of regular expressions.
[]	A bracket expression. Matches a single character that is contained within the brackets.	[abc] matches "" a "", "" b "", or "" c ". [a-z] specifies a range which matches any lowercase letter from "" a "" to "" z ". These forms can be mixed: [abcx-z] matches

Metacharacter	Description	Example
		<code>"a", "b", "c", "x", "y", or "z", as does [a-cx-z].</code> The - character is treated as a literal character if it is the last or the first (after the ^, if present) character within the brackets: [abc-], [-abc]. Backslash escapes are not allowed. The] character can be included in a bracket expression if it is the first (after the ^) character: []abc].
<code>[^]</code>	Matches a single character that is not contained within the brackets.	[^abc] matches any character other than "a", "b", or "c". [^a-z] matches any single character that is not a lowercase letter from "a" to "z". Likewise, literal characters and ranges can be mixed.
<code>^</code>	Matches the starting position within the string.	—
<code>\$</code>	Matches the ending position of the string or the position just before a string-ending newline.	—
<code>()</code>	Defines a subexpression.	—
<code> </code>	The choice (also known as alternation or set union) operator matches either the expression before or the expression after the operator.	abc def matches "abc" or "def".
<code>\b</code>	Matches a zero-width boundary between a word-class character (see next) and either a non-word class character or an edge.	—
<code>\w</code>	Matches an alphanumeric character, including " _"	—
<code>\W</code>	Matches a non-alphanumeric character, excluding " _"	—
<code>\s</code>	Matches a whitespace character, like tab, line feed, form feed, carriage return, and space.	—
<code>\S</code>	Matches anything BUT a whitespace.	—
<code>\d</code>	Matches a digit.	—
<code>\D</code>	Matches a non-digit.	—

Metacharacter	Description	Example
(? _{subpattern})	The characters following the assertion must match subpattern, but no characters are consumed.	—
(?! _{subpattern})	The characters following the assertion must not match subpattern, but no characters are consumed.	—
(?: _{subpattern})	Defines a subexpression. Does not create a backreference.	—
\$1, \$2, ...	References the backreference with given number.	—

Note

To search for a backslash (\) and replace it with another character, type in four backslashes, \\\\.

This is because the *SimTalk* parser uses the backslash as an **escape character**.

```
// replaces the backslash with an empty space
regex_replace("my\\text", "\\\\", " ")

print regex_replace("my\\text", "\\\\", " ")
-- outputs my text
```

Type

Function

Syntax

```
regex_search(Text:string, RegularExpression:string) → string
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text which you want to search for the *regular expression*.
- The parameter *RegularExpression* of data type *string* designates the *regular expression* that you want to find.

Return Value

The return value has the data type *string*.

Is the *string* which first matches the expression.

Is an empty string "" if *Plant Simulation* does not find the search *string*.

Examples

```
// extracts the time portion
regex_search("12345678 11:53am test", "((1[0-2])|(0?[1-9])):([0-5][0-9])
((am)|(pm))")
-- returns "11:53am"
```

```
// extracts a HTML group
regex_search("thrth <h1>123123</h1> etheth 123", "<(.*?)>(.*?)</(\1)>")
-- returns <h1>123123</h1>
```

See also

[regex_replace \[SimTalk\]](#)

[regex_search2 \[SimTalk\]](#)

regex_search2 [SimTalk]

Searches for the specified *Text* in the passed text that matches the *RegularExpression*.

Remarks

The function *regex_search2* corresponds to the function *regex_search*. As opposed to it the return value of *regex_search2* is an *array* of data type *string* containing all found partial expressions, followed by the preceding text, followed by the text at the end.

Note

To search for a backslash (\) and replace it with another character, type in four backslashes, \\\\.

This is because the *SimTalk* parser uses the backslash as an **escape character**.

```
// replaces the backslash with an empty space
regex_replace("my\\text", "\\\\", " ")

print regex_replace("my\\text", "\\\\", " ")
-- outputs my text
```

Type

Function

Syntax

```
regex_search2(Text:string, RegularExpression:string) → string[]
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text which you want to search for the *regular expression*.
- The parameter *RegularExpression* of data type *string* designates the *regular expression* that you want to find.

Return Value

The return value is an *array* of data type *string*.

Examples

```
print regex_search2("John.Doe@acme.com", "(\w*)\.(\\w*)@(\w*)\.(\\w*)")  
[John.Doe@acme.com, John, Doe, acme, com, , ]
```

```
print regex_search2("EMail:John.Doe@acme.com [John Doe]", ":(\\w*)\\.  
(\\w*)@(\w*)\.(\\w*)")  
[:John.Doe@acme.com, John, Doe, acme, com, EMail, [John Doe]]
```

See also

[regex_replace \[SimTalk\]](#)

[regex_search \[SimTalk\]](#)

splitString [SimTalk]

Splits a *string* into its individual *string components*.

Type

Function

Syntax

```
splitString(Text:string, Delimiter:string[, KeepEmptyItems:boolean:=false])  
→ string[]
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the *string* which you want to split.
- The parameter *Delimiter* of data type *string* designates the delimiter that was used in the parameter *Text* to separate its components.

Note

The *delimiter* can be any character.

If the parameter *Delimiter* is empty, the result/return value is an *array* of the individual letters of the text that is to be split up.

- The optional parameter *KeepEmptyItems* of data type *boolean* sets if empty strings are to be incorporated into the array (`true`) or not (`false`).

Default Value of the Parameter

The default value is `false`.

Return Value

The return value is an *array* of data type *string*.

Examples

```
print splitString("a,b,c,d", ",")      -- returns [a, b, c, d] in the  
Console
```

```
print splitString("one,two;three;four,five", ",;")  
-- returns [one, two, three, four, five] in the Console
```

```
print splitString("a;b;;c", ";", true) -- returns [a, b, , c] in the Console  
print splitString("a;b;;c", ";")      -- returns [a, b, c]
```

```
var text:string := "Hello World"  
var letters:string[] := splitString(text, "")  
print letters  
-- returns [H, e, l, l, o, , W, o, r, l, d]
```

splitStringToNum [SimTalk]

Splits a *string* into an *array* of numbers of data type *real*.

Type

Function

Syntax

```
splitStringToNum(Text:string, Delimiter:string) → real
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the *real* numbers which you want to split.
- The parameter *Delimiter* of data type *string* designates the delimiter that was used in the parameter *Text* to separate its components.

Return Value

The return value is an *array* of data type *real*.

Example

```
print splitStringToNum("1.2;2.8!3.4;4.5;6.8!3.6", ";!")  
// returns [1.2, 2.8, 3.4, 4.5, 6.8, 3.6] in the Console
```

strAscii [SimTalk]

Returns the Unicode character code of the specified character of a string.

Remarks

If you do not specify the optional parameter *PositionOfCharacter*, the function returns the character code of the first character of the string.

Type

Function

Syntax

```
strAscii(String:string[, PositionOfCharacter:integer]) → integer
```

Parameters

You can specify the following parameters:

- The parameter *String* of data type *string* designates the string.
- The optional parameter *PositionOfCharacter* of data type *integer* designates the position of the character whose character code you want to know.

The number 1 is the first character. The range of values varies from 1 to the length of the string.

Specify a negative value to receive the n-th character starting at the end of the string.

Return Value

The return value has the data type *integer*.

Is a number between 1 and 65535.

Examples

```
var s: string := "Hello World!"
for var i := 1 to strlen(s)
    print strChr(strAscii(s, i)), ", ", strAscii(s, i)
next
```

```
print strAscii("Test", -1) // returns 116, the Ascii-Code of the letter 't'
```

See also

[strChr \[SimTalk\]](#)

strChr [SimTalk]

Returns a string of length 1 that consists of the character with the given Unicode character code.

Type

Function

Syntax

```
strChr(UnicodeNumber:integer) → string
```

Parameter

The parameter *UnicodeNumber* of data type *integer* designates the Unicode character code. This comes in handy if you want to create strings that contain special characters that you cannot enter on your keyboard.

Return Value

The return value has the data type *string*.

Examples

```
print "cost:"+strChr(9)+"$42" // strChr(9) returns a tab
```

```
print strChr(20616) // prints the Chinese character 栗
```

See also

[strAscii \[SimTalk\]](#)

strCopy [SimTalk] - string segments

Copies the specified segment of a string and returns it.

Type

Function

Syntax

```
strCopy(String:string, Position:integer, NumberOfCharacters:integer)
```

Parameters

You can specify the following parameters:

- The parameter *String* of data type *string* designates the string from which *Plant Simulation* copies the segment.
- The parameter *Position* of data type *integer* designates the position from which on *Plant Simulation* copies the segment.

Plant Simulation does not display an error message if the *Position* is less than 1 or the *NumberOfCharacters* is greater than the length of the parameter of data type *string*.

- The parameter *NumberOfCharacters* of data type *integer* designates the number of characters you would like to *strCopy*.

Example

```
print strCopy("abcdef",2,3)    // returns "bcd"  
print strCopy("abcdef",4,10)  // returns "def"  
print strCopy("abcdef",-1,4)   // returns "ab" (-1 and 0 letter are counted)
```

strIncl [SimTalk]

Inserts the specified text into another string and returns the combined text of both.

Type

Function

Syntax

```
strIncl(TextToBeInserted:string, Text:string, Position:integer) → string
```

Parameters

You can specify the following parameters:

- The parameter *TextToBeInserted* of data type *string* designates the text which you want to insert into the other string.
- The parameter *Text* of data type *string* designates the text into which the *string* will be inserted.
- The parameter *Position* of data type *integer* designates the position in front of which the *TextToBeInserted* will be inserted.

The first character of *TextToBeInserted* has the index 1. *Plant Simulation* does not show an error message if the parameter *Position* is less than 1 or if the *Position* is greater than the length of the string. *TextToBeInserted* will then be inserted before or after *Text* respectively.

Return Value

The return value has the data type *string*.

Example

```
print strIncl("XYZ","abcdef",3) // returns "abXYZcdef"
print strIncl("XYZ","abcdef",-1) // returns "XYZabcdef"
print strIncl("XYZ","abcdef",1) // returns "XYZabcdef"
print strIncl("XYZ","abcdef",20) // returns "abcdefXYZ"
```


strLen [SimTalk]

Returns the length, i.e., the number of characters of the specified string.

Type

Function

Syntax

```
strLen(Text:string) → integer
```

Parameter

The parameter *Text* of data type *string* designates the string.

Return Value

The return value has the data type *integer*.

Example

```
print strLen("shop") // returns 4
```

strLPos [SimTalk]

Returns the position at which the designated text occurs for the first time within the specified string.

Type

Function

Syntax

```
strLPos(StrToSearchFor:string, StrToSearchIn:string) → integer
```

Parameters

You can specify the following parameters:

- The parameter *StrToSearchFor* of data type *string* designates the *string* whose position you would like to know.
- The parameter *StrToSearchIn* of data type *string* designates the *string* in which the first occurrence of the *string* to be searched for is searched.

Return Value

The return value has the data type *integer*.

Is 0 if the search *string* was not found.

Example

```
print strLPos("b","abcdefb") // returns 2
```

strOmit [SimTalk]

Copies the specified *string* and deletes a succession of characters from it.

Type

Function

Syntax

```
strOmit(SourceText:string, Position:integer, NumberOfCharacters:integer) → string
```

Parameters

You can specify the following parameters:

- The parameter *SourceText* of data type *string* designates the string which you want to copy.
- The parameter *Position* of data type *integer* designates the position within the *string* from which on *Plant Simulation* deletes characters.
- The parameter *NumberOfCharacters* of data type *integer* designates the number of characters which *Plant Simulation* deletes.

Return Value

The return value has the data type *string*.

Example

```
print strOmit("abcdef",3,2) // returns "abef"  
print strOmit("abcdef",0,3) // returns "cdef" (0 letter is counted)
```

strRcopy [SimTalk]

Copies a number of characters within the *string*, starting from the right.

Type

Function

Syntax

```
strRcopy(SourceText:string, NumberOfCharacters:integer) → string
```

Parameters

You can specify the following parameters:

- The parameter *SourceText* of data type *string* designates the string from which you want to copy the characters.
- The parameter *NumberOfCharacters* of data type *integer* designates the number of characters to be copied, starting from the right.

Return Value

The return value has the data type *string*.

Example

```
print strRcopy("abcde",2) // returns de
```

strReplace [SimTalk]

Searches for the text you specified as the *TextToFind* within the string *SourceText* and replaces all occurrences with the text you specified as the string *ReplaceBy*. The function returns the resulting *string*.

Type

Function

Syntax

```
strReplace(SourceText:string, TextToFind:string, ReplaceBy:string) -> string
```

Parameters

You can specify the following parameters:

- The parameter *SourceText* of data type *string* designates the source text within which you want to search.
- The parameter *TextToFind* of data type *string* designates the text which you want to find.
- The parameter *ReplaceBy* of data type *string* designates the text which is going to replace the text you are searching for in all occurrences, which it found.

Return Value

The return value has the data type *string*.

Plant Simulation does not modify the *SourceText* during and after the search.

Example

```
var SourceText : string := "hello world"
var Result1 : string := strReplace(SourceText, "l", "XY") -- returns
"heXYXYo worXYd"
var Result2 : string := strReplace(SourceText, "ll", "x") -- returns "hexo
world"
var Result3 : string := strReplace(SourceText, "L", "x") -- returns "hello
world" because upper-case L is not contained
```

strRpos [SimTalk]

Returns the position at which the designated text occurs for the last time within the *SourceText*.

Type

Function

Syntax

```
strRpos(TextToBeFound:string, SourceText:string) → integer
```

Parameters

You can specify the following parameters:

- The parameter *TextToBeFound* of data type *string* designates the text whose position you want to query.
- The parameter *SourceText* of data type *string* designates the string within which you want to query.

Return Value

The return value has the data type *integer*.

Is 0 if no match was found.

Example

```
print strRpos("bb","abbabbbaaaba") // returns 5
```

strToHtml [SimTalk]

Replaces all occurrences of characters, which have a syntactic meaning in HTML, within the specified text with the respective HTML entities to create valid HTML code.

Remarks

This way you can enter letters, that are syntax-specific to HTML, as the function transforms them to readable letters.

Type

Function

Syntax

```
strToHtml(Text:string) → string
```

Parameter

The parameter *Text* of data type *string* designates the text that you want to convert to valid HTML.

Return Value

The return value has the data type *string*.

Examples

```
-- replaces the greater-than-sign > with the HTML entity &gt;  
var s: string := strToHtml("var > 4") -- results in var &gt; 4
```

```
-- replaces the double-quotation marks " with the respective HTML entity  
&quot;  
print strToHtml("This is a \"quotation\"") -- results in This is a  
&quot;quotation&quot;
```

strToLower [SimTalk]

Changes upper-case letters within the specified *string* to lower-case letters.

Type

Function

Syntax

```
strToLower(UpperCaseLetter:string) → string
```

Parameter

The parameter *UpperCaseLetter* of data type *string* can consist of any combination of upper-case letters, lower-case letters, and special characters.

The resulting *string* then only contains lower-case letters and special characters.

Return Value

The return value has the data type *string*.

Examples

```
print strToLower("SHANIA")           // returns "shania"
```

```
print strToLower("Hello World!")     // returns "hello world!"
```


strToUpper [SimTalk]

Changes lower-case letters in the specified *string* to upper-case letters.

Type

Function

Syntax

```
strToUpper(LowerCaseLetters:string) → string
```

Parameter

The parameter *LowerCaseLetters* of data type *string* can consist of any combination of upper-case letters, lower-case letters, and special characters.

The resulting *string* then only contains upper-case letters and special characters.

Return Value

The return value has the data type *string*.

Examples

```
print strToUpper("shania")           // returns "SHANIA"
```

```
print strToUpper("Hello World!")     // returns "HELLO WORLD!"
```